OS Unit 4 Orange Problem

PES-VCS — Building a Version Control System from Scratch

Name	SIMON C A
SRN	PES2UG24CS503
Course	Operating Systems
Platform	Ubuntu 22.04 (VirtualBox)
Language	C (GCC, libcrypto)

Project Overview

PES-VCS is a local version control system built in C on Linux, modeled on Git's internal architecture. The project implements content-addressable object storage, tree serialization, a text-based staging index, and a commit graph with parent pointers. Every component maps directly to OS and filesystem concepts: atomic writes using temp-file-then-rename, SHA-256 integrity hashing, directory sharding, linked structures on disk, and reference-pointer updates.

The system supports five CLI commands: **init**, **add**, **status**, **commit**, and **log**. All object data lives under **.pes/objects/** sharded by the first two hex characters of each object's SHA-256 hash. Branch pointers reside in **.pes/refs/heads/** and HEAD tracks the current branch.

Phase 1: Object Storage Foundation

Phase 1 implements the content-addressable object store — the core of PES-VCS. Two functions were implemented in **object.c**:

object_write — Prepends a type header (*blob <size>\0*), computes SHA-256 of the full object, and performs an atomic write using the temp-file-then-rename pattern into a sharded subdirectory.

object_read — Reads the file, parses the type/size header, recomputes the SHA-256 hash and compares it to the filename to verify integrity, then returns the raw data portion.

Key OS concepts: atomic writes (fsync + rename), SHA-256 hashing, directory sharding for inode scalability.

Screenshot 1A — Phase 1 Test Results

```
[04/21/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ export PES_AUTHOR="SIMON C A <PES2UG24CS503>"
[04/21/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
[04/21/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Figure 1A: ./test_objects — all Phase 1 tests passing (blob storage, deduplication, integrity check)

Screenshot 1B — Object Store Structure

```
[04/21/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
[04/21/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$
```

Figure 1B: find .pes/objects -type f — sharded directory layout (objects stored under 2-char prefix directories)

Phase 2: Tree Objects

Phase 2 implements tree serialization in **tree.c**. A tree object represents a directory snapshot as a sorted list of entries, each with a mode, SHA-256 hash, and filename. The key function implemented is **tree_from_index**, which builds a tree hierarchy from the staging index, handling nested paths (e.g., *src/main.c* creates a *src* subtree), writes all tree objects to the object store, and returns the root tree hash.

Trees are serialized deterministically: entries are sorted by name so the same set of files always produces the same tree hash — a requirement for deduplication and correctness.

Screenshot 2A — Phase 2 Test Results

```
[04/22/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Figure 2A: ./test_tree — all Phase 2 tests passing (serialize/parse roundtrip, deterministic serialization, 139 bytes)

Screenshot 2B — Raw Tree Object (xxd)

```
[04/22/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ xxd .pes/objects/2c
/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 |
head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a        blob 6.hello.
```

Figure 2B: xxd of a raw tree object — shows binary format starting with 'blob 6.hello.' type header

Phase 3: The Index (Staging Area)

Phase 3 implements the staging area in **index.c**. The index is a human-readable text file at **.pes/index** with one entry per line in the format: *<mode> <hash-hex> <mtime> <size> <path>*.

Three functions were implemented:

index_load — Reads the index file into an Index struct; if the file doesn't exist, initializes an empty index (not an error).

index_save — Sorts entries by path, writes to a temp file, calls fsync(), then renames atomically to .pes/index to prevent corruption on crash.

index_add — Reads the target file, writes its content as a blob object, and inserts or updates the index entry with the new hash, mtime, and size.

Screenshot 3A — pes status Output

```
[04/22/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ ./pes status
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  Makefile
  untracked:  index.h
  untracked:  tree.c
  untracked:  test_tree
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  commit.c
  untracked:  file1.txt
  untracked:  index.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  file2.txt
  untracked:  commit.h
  untracked:  pes.h
```

Figure 3A: ./pes status — shows staged changes, unstaged changes, and untracked files after init and add

Screenshot 3B — Index File Contents

```
[04/22/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db
9ebb4 1776832563 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f6061
99f7f 1776832563 6 file2.txt
```

Figure 3B: cat .pes/index — human-readable index with mode, hash, mtime, size, and filename for each staged file

Phase 4: Commits and History

Phase 4 implements commit creation in **commit.c** via **commit_create**. This function: builds a tree from the current index using tree_from_index(), reads the current HEAD as the parent commit hash (absent for the initial commit), obtains the author string from pes_author() (sourced from the PES_AUTHOR environment variable), serializes the commit object, writes it to the object store, and atomically updates the branch ref so HEAD advances.

Commits form a singly-linked list on disk: each commit records its parent's hash, enabling pes log to walk history by following parent pointers until the chain terminates.

Screenshot 4A — pes log (Three Commits)

```
commit 3d77a86ca0ff70cfb769fcafe873050ea30e6e778b5e62e201398ce2a93
f21cd
Author: SIMON C A <PES2UG24CS503>
Date:   1776833238

    Add farewell

commit 5258812452a2d24c37aa9a2afbf64d0f3cf32f0770e18b0febe613f96fd
64bc8
Author: SIMON C A <PES2UG24CS503>
Date:   1776833237

    Add world

commit f3384cab9ccebdcf61ab64a43c453542abc0b7fa57f3dfe5167ff68e369
6a9cf
Author: SIMON C A <PES2UG24CS503>
Date:   1776833237

    Initial commit
```

Figure 4A: ./pes log — three commits (Initial commit, Add world, Add farewell) with full hashes, author SIMON C A, and timestamps

Screenshot 4B — Object Store After Three Commits

```
[04/22/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ find .pes -type f |
 sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21
cbd6c05e5f43
.pes/objects/3d/77a86ca0ff70cfb769fcafe873050ea30e6e778b5e62e20139
8ce2a93f21cd
.pes/objects/52/58812452a2d24c37aa9a2afbf64d0f3cf32f0770e18b0febe6
13f96fd64bc8
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f
4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934
310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff1
31f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382
967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df83443888
0e13a68652bc
.pes/objects/f3/384cab9ccebdcf61ab64a43c453542abc0b7fa57f3dfe5167f
f68e3696a9cf
.pes/refs/heads/main
```

Figure 4B: find .pes -type f | sort — full object store showing HEAD, index, refs, and all blob/tree/commit objects

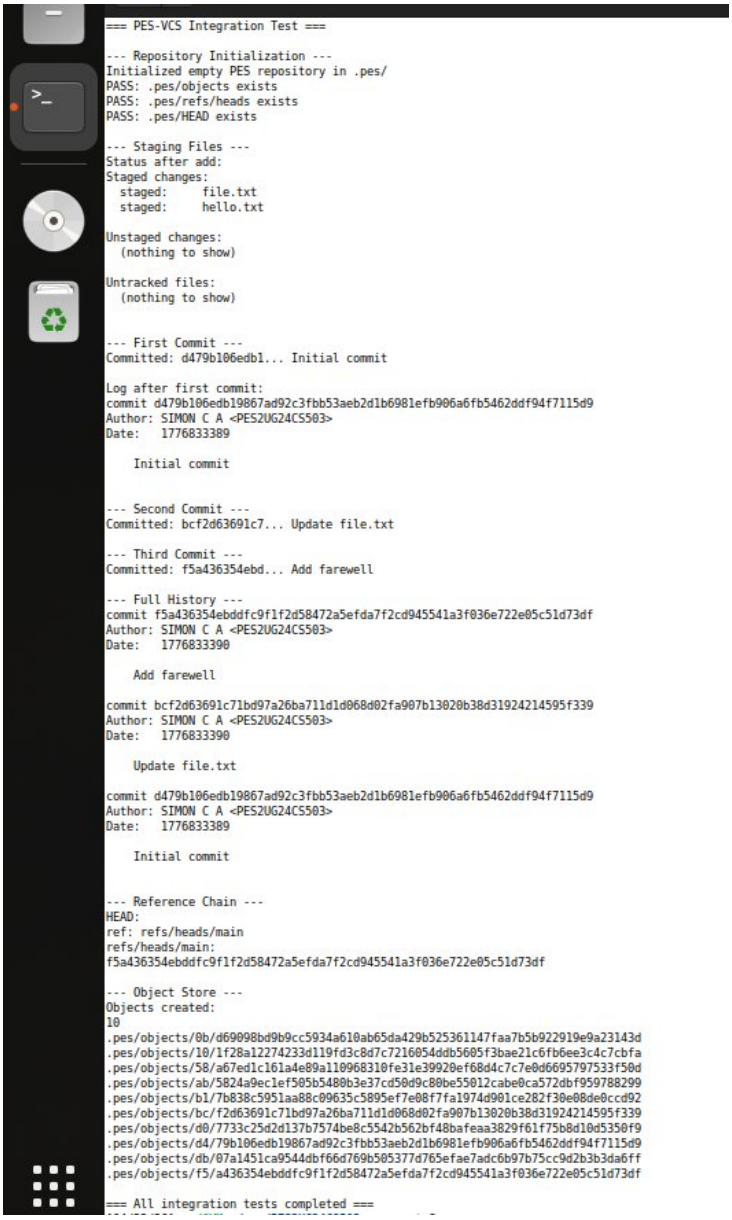
Screenshot 4C — Reference Chain

```
[04/22/26]seed@VM:~/.../PES2UG24CS503-pes-vcs$ cat .pes/refs/heads
/main
cat .pes/HEAD
3d77a86ca0ff70cfb769fcafe873050ea30e6e778b5e62e201398ce2a93f21cd
ref: refs/heads/main
```

Figure 4C: cat .pes/refs/heads/main (latest commit hash) and cat .pes/HEAD (ref: refs/heads/main)

Integration Test

The full integration test (test_sequence.sh) exercises the complete PES-VCS workflow end-to-end: repository initialization, staging files, making multiple commits, log inspection, and verifying the reference chain and object store integrity.



```
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: d479b106edb1... Initial commit

Log after first commit:
commit d479b106edb19867ad92c3fbb53aeb2d1b6981efb906a6fb5462ddf94f7115d9
Author: SIMON C A <PES2UG24CS503>
Date:   1776833389

    Initial commit


--- Second Commit ---
Committed: bcf2d63691c7... Update file.txt

--- Third Commit ---
Committed: f5a436354ebd... Add farewell

--- Full History ---
commit f5a436354ebddfc9f1f2d58472a5efda7f2cd945541a3f036e722e05c51d73df
Author: SIMON C A <PES2UG24CS503>
Date:   1776833390

    Add farewell

commit bcf2d63691c71bd97a26ba711d1d068d02fa907b13020b38d31924214595f339
Author: SIMON C A <PES2UG24CS503>
Date:   1776833390

    Update file.txt

commit d479b106edb19867ad92c3fbb53aeb2d1b6981efb906a6fb5462ddf94f7115d9
Author: SIMON C A <PES2UG24CS503>
Date:   1776833389

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f5a436354ebddfc9f1f2d58472a5efda7f2cd945541a3f036e722e05c51d73df

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/bc/f2d63691c71bd97a26ba711d1d068d02fa907b13020b38d31924214595f339
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/d4/79b106edb19867ad92c3fbb53aeb2d1b6981efb906a6fb5462ddf94f7115d9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f5/a436354ebddfc9f1f2d58472a5efda7f2cd945541a3f036e722e05c51d73df

=== All integration tests completed ===
```

Figure 4D: Full integration test output — all phases pass including init, add, commit, log, reference chain, and object store verification

Analysis Questions — Phase 5: Branching and Checkout

Q5.1 — Implementing pes checkout

A branch is simply a file in **.pes/refs/heads/** containing a commit hash. To implement **pes checkout <branch>**, the following must happen:

Files changed in .pes/: HEAD must be updated to contain *ref: refs/heads/<branch>* — this is the only metadata change required if the branch already exists.

Working directory update: The new branch's tip commit must be read, its tree hash extracted, and then all files tracked in that tree must be written to the working directory. Files present in the current tree but absent from the target tree must be deleted. This requires recursively resolving tree objects from the object store and writing each blob to the correct path.

What makes it complex: (1) The operation must be atomic — a crash mid-checkout leaves the working directory inconsistent. (2) Uncommitted changes must be detected and refused to avoid data loss (dirty-tree check). (3) Subdirectory creation and deletion must be handled carefully. (4) File permissions (mode bits) must be restored from the tree entries. (5) Symbolic links and executable flags add further edge cases.

Q5.2 — Detecting Dirty Working Directory Conflicts

To detect whether a file would conflict during branch switch, three snapshots must be compared: the current branch's HEAD tree, the target branch's HEAD tree, and the current index.

The algorithm is: for every file that differs between the source and target tree, look up that file in the index. If the index hash differs from the working directory hash (computed on the fly by reading the actual file and hashing it), the file is dirty and checkout must be refused.

Concretely: (1) Resolve both branches' root tree objects from the object store. (2) Build a set of file paths that differ between the two trees. (3) For each such path, read the actual file from disk, compute its blob hash, and compare to the index entry's stored hash. If they differ, the file has uncommitted modifications and checkout must abort with an error listing the conflicting files.

This uses *only* the index (for the last-staged hash) and the object store (for tree resolution) — no additional metadata is needed.

Q5.3 — Detached HEAD State

In detached HEAD state, **.pes/HEAD** contains a raw commit hash rather than a branch reference like *ref: refs/heads/main*. Commits made in this state create new commit objects correctly, but **no branch pointer advances** — the new commits are reachable only from HEAD itself.

If the user then checks out a branch, HEAD is overwritten with the branch reference, and the previously made commits become unreachable from any named ref. They are not immediately deleted but will be collected by the next garbage collection pass that sweeps for unreachable objects.

Recovery: If the commit hashes are still known (e.g., from terminal history or a reflog), the user can create a new branch pointing to the last detached commit: *pes branch recover-branch <hash>*. Git additionally maintains a reflog (a log of all HEAD movements) which makes recovery straightforward

even without remembering the hash. PES-VCS as implemented does not have a reflog, so recovery depends entirely on the user having noted the commit hash before switching away.

Analysis Questions — Phase 6: Garbage Collection

Q6.1 — Finding and Deleting Unreachable Objects

The garbage collection algorithm is a mark-and-sweep traversal of the object graph:

Step 1 — Enumerate roots: Collect all commit hashes that are directly pointed to by named references: every file under **.pes/refs/** and HEAD itself (if in detached state). These are the GC roots.

Step 2 — Mark reachable objects (BFS/DFS): Starting from each root commit hash, traverse the object graph: add the commit hash to a reachable set, read the commit to get its tree hash and parent hash, add those to the set, then recursively resolve tree objects (adding each blob and subtree hash). Continue until the entire transitive closure is marked.

Step 3 — Sweep: Enumerate all files under **.pes/objects/**. Reconstruct the full hash from the directory name + filename. If the hash is not in the reachable set, delete the file. Empty shard directories can also be removed.

Data structure: A hash set (unordered_set in C++, or a hash table in C) is ideal for O(1) lookup during the sweep phase. Alternatively, a sorted array with binary search gives O(log n) lookup.

Scale estimate for 100,000 commits, 50 branches: Each commit references one tree and one parent. A typical tree might reference 20–100 blobs. Assuming average 50 blobs per commit and reasonable deduplication (say 30% unique blobs), the total objects are approximately: 100,000 commits + 100,000 trees + ~1,500,000 unique blobs = ~1.7 million objects to visit during marking. The sweep then scans all files on disk once, which is I/O-bound.

Q6.2 — Race Condition Between GC and Commit

A dangerous race condition arises if GC runs concurrently with a commit operation:

The race: The commit process first writes new blob objects (step A), then writes the tree object referencing those blobs (step B), then writes the commit object (step C), then updates the branch ref (step D). If GC runs its mark phase between steps A and D, the newly written blobs are not yet reachable from any named ref — they exist in the object store but no commit or tree points to them yet. GC will mark them as unreachable and delete them. When the commit process reaches step B and tries to create a tree referencing those blobs, or when a future read tries to retrieve them, they are gone — corrupting the repository.

How Git avoids this: Git's real GC uses a grace period — it refuses to delete any object created or modified within the last 14 days (configurable via gc.pruneExpire), giving in-progress operations time to complete. It also uses file-based locking (the gc.pid lock file) to prevent concurrent GC runs. For a correct implementation in PES-VCS, one approach is to take a write lock on the object store directory before the GC sweep phase, or to implement the grace period by checking object file mtime before deletion.